

APAI LAB 05: Tiny Transformers on PULP Platforms

Călin Diaconu – calin.diaconu2@unibo.it
Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it
Angelo Anzalone - angelo.anzalone4@unibo.it
Ihor Sachok – ihor.sachok@unibo.it
Luca Parigi – luca.parigi2@unibo.it
Marco La Barbera – marco.labarbera2@unibo.it



01. Lab Information

01. Lab Information – Objective of the Class

- **Subjects**:
  - Attention mechanism on the PULP architecture
  - Llama Transformer
  - Profiling code execution
- **Programming language**: C
- **Lab duration**: 3h

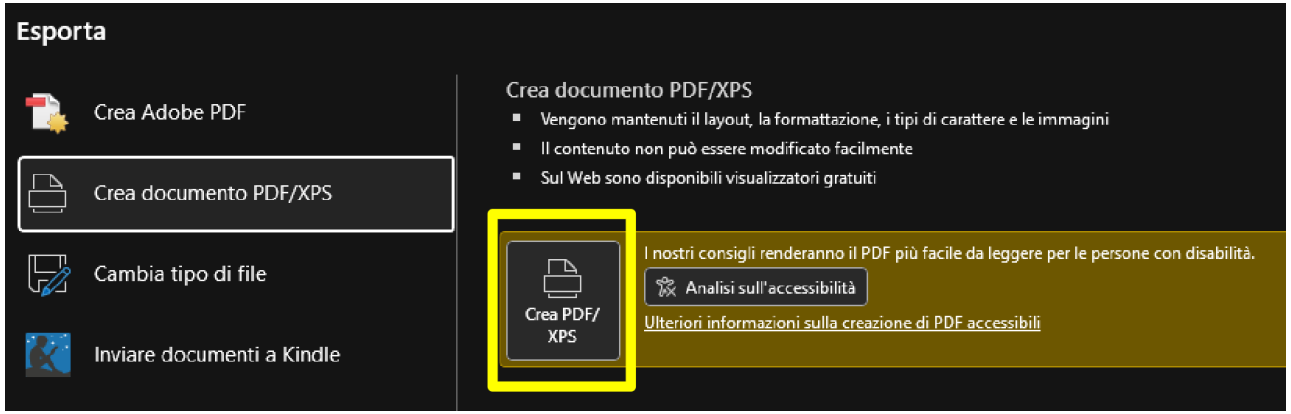
- **AI policy**: we will not prevent you from using AI in solving the assignments, but the lab is meant to be a hands-on learning experience. To keep it useful, we have included an .md file (also used in a Stanford class), that should be automatically read by AI agents, in order to prevent them from giving you a direct solution and rather guide you towards finding it yourself.

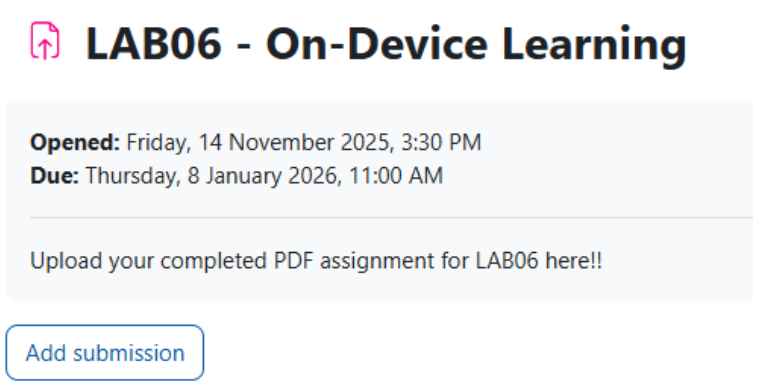
- The class is meant to be interactive: coding together and on your own!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

01. Lab Information – How to Deliver the Assignment

- You will deliver **ONLY THE PDF** assignment, **NO code**
  - Download the **deliverable template** from Virtuale
  - **Fill in the results** required by the assignment
  - **Export to PDF** format (this option is available in most document editors – Google Docs, Office Word, or as a "Print to PDF" option in the Print menu)
  - **Rename the file** to *LAB<number_of_the_lesson>_APAI_<name_surname>.pdf*
  - Use the **Virtuale platform to upload the PDF** before the **deadline (Sunday 15th of November 2026, end of day, 23:59)**
  - For each lab that you solve and submit successfully, you will receive **1 extra point**, up to a **maximum of 4 points**





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



02. Prerequisites

02. Prerequisites – Containers & Docker



STEPS ONLY FOR PERSONAL MACHINES

- **What is a Container?**
  - A lightweight, portable, and self-contained environment that bundles together
    - A consistent runtime environment
    - Your application code
    - All its dependencies (libraries, tools, configurations)
- **What is Docker?**
  - The most used platform for working with containers, which allows the user to:
    - **Build images**, defined in a Dockerfile
    - **Run containers** – isolated environments, created according to the "recipe" defined inside an image
    - **Share images with other people** through the Docker Hub, for reuse and consistency in development
- The **Docker image** used in this course includes the PULP-SDK and its related compilation toolchain

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Docker Setup (WSL install)



STEPS ONLY FOR PERSONAL MACHINES

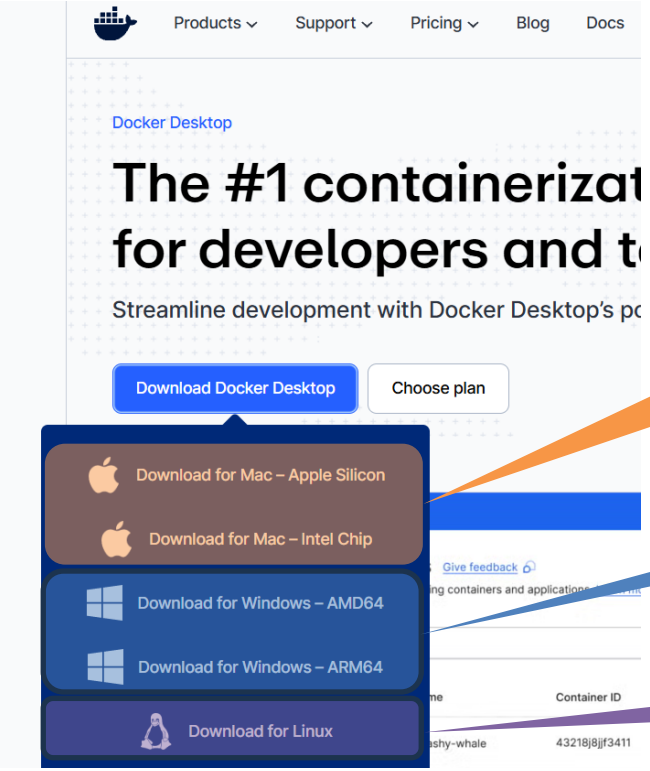
- **Step 0** (for **Windows users only!**) – install Windows Subsystem for Linux (WSL 2)
  - **What is WSL?**
    - **A compatibility layer** that lets you run a genuine Linux environment directly on Windows, without a separate virtual machine or dual boot
    - Gives access to Linux command-line tools and applications, running natively alongside your Windows apps
    - It can run both command-line and graphical Linux apps directly inside Windows
  - The **advantage** of running Docker through WSL is that, while in both cases Docker runs through a virtual machine, the WSL is much lighter and it doesn't slow down the entire system
  - If you plan to work on the lab on a **Windows** machine, please **follow these installation instructions for WSL**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Docker Setup

STEPS ONLY FOR PERSONAL MACHINES

- **Step 1 – install Docker Desktop**
  - Choose, download and install the proper version from the official website.



- Apple Mac versions
- Choose based on your computer's processor

- Windows versions (AMD64 for most, unless you are **VERY SURE** that your processor is ARM64)
- **CHOOSE THIS EVEN IF YOU WORK WITH WSL!**

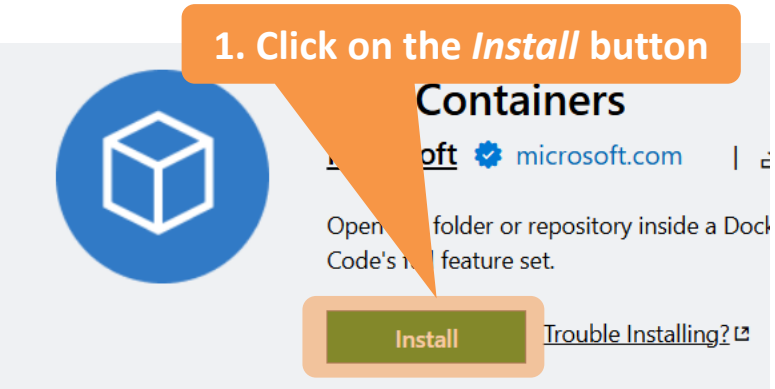
- Universal Linux version
- **If working with WSL, choose a Windows version!**

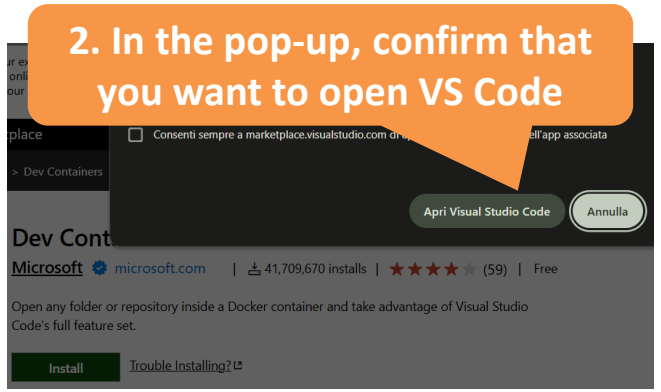
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

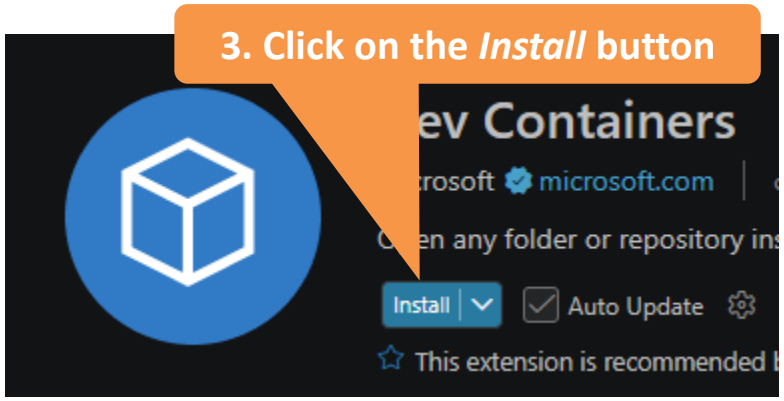
02. Prerequisites – VS Code

STEPS ONLY FOR PERSONAL MACHINES

- **Step 2 – Install Visual Studio (VS) Code**
  - **What is VS Code?**
    - A free, popular and versatile text editor from Microsoft
    - It is highly customizable through a large extension library
    - The one that interests us is the **Dev Containers** extension, that enables VS Code to work with Containers
  - **How to install?**
    - Choose, download and install for your operating system from the official website
    - **Once VS Code is installed**, we need to install the extension for working with Containers
      - Find and install it from the VS Code Marketplace







ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Error Solving

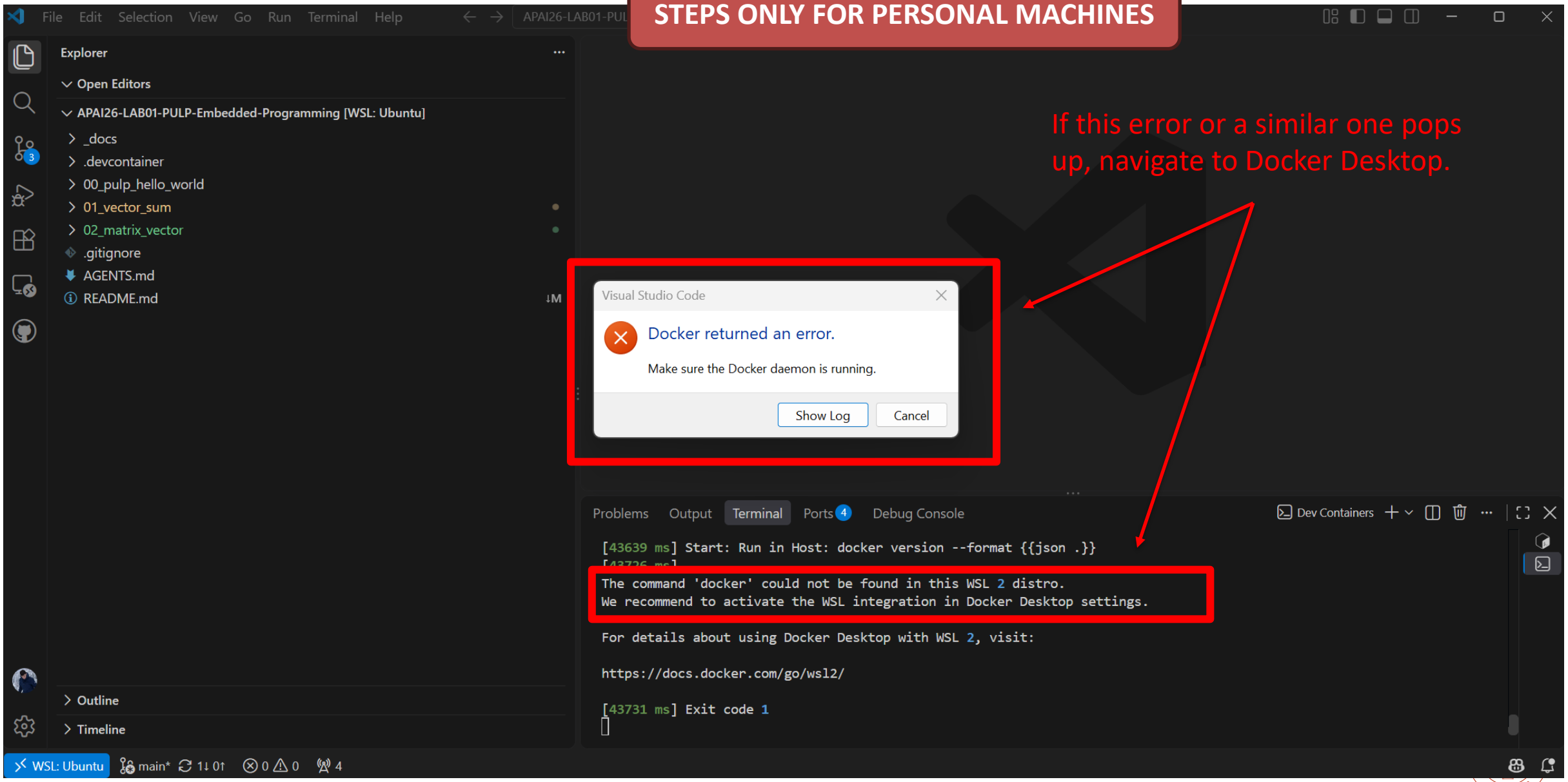
STEPS ONLY FOR PERSONAL MACHINES

IN VS CODE, IF DOCKER RETURNS AN ERROR,

FOLLOW THE INSTRUCTIONS IN THE FOLLOWING SLIDES

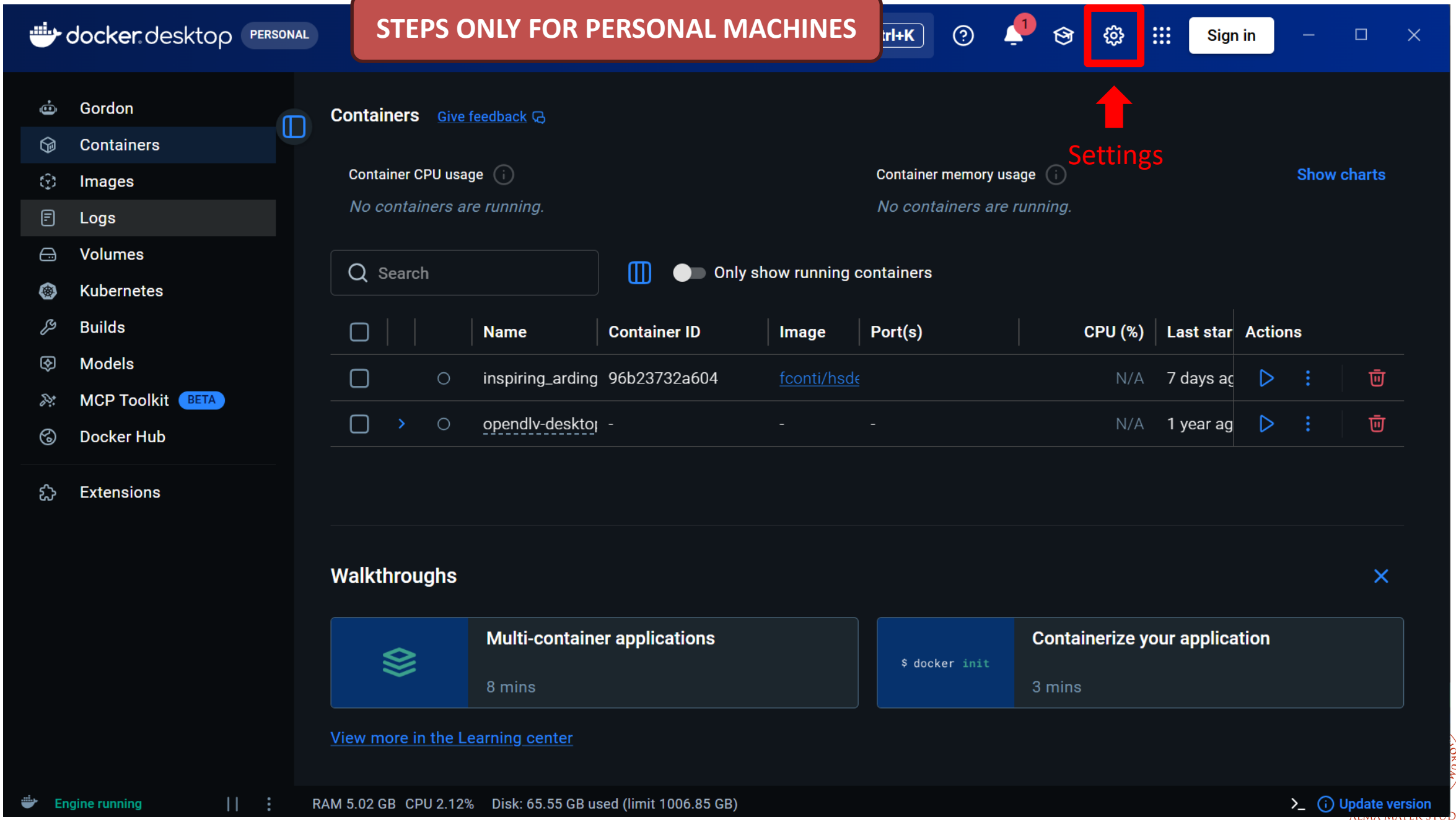
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

STEPS ONLY FOR PERSONAL MACHINES

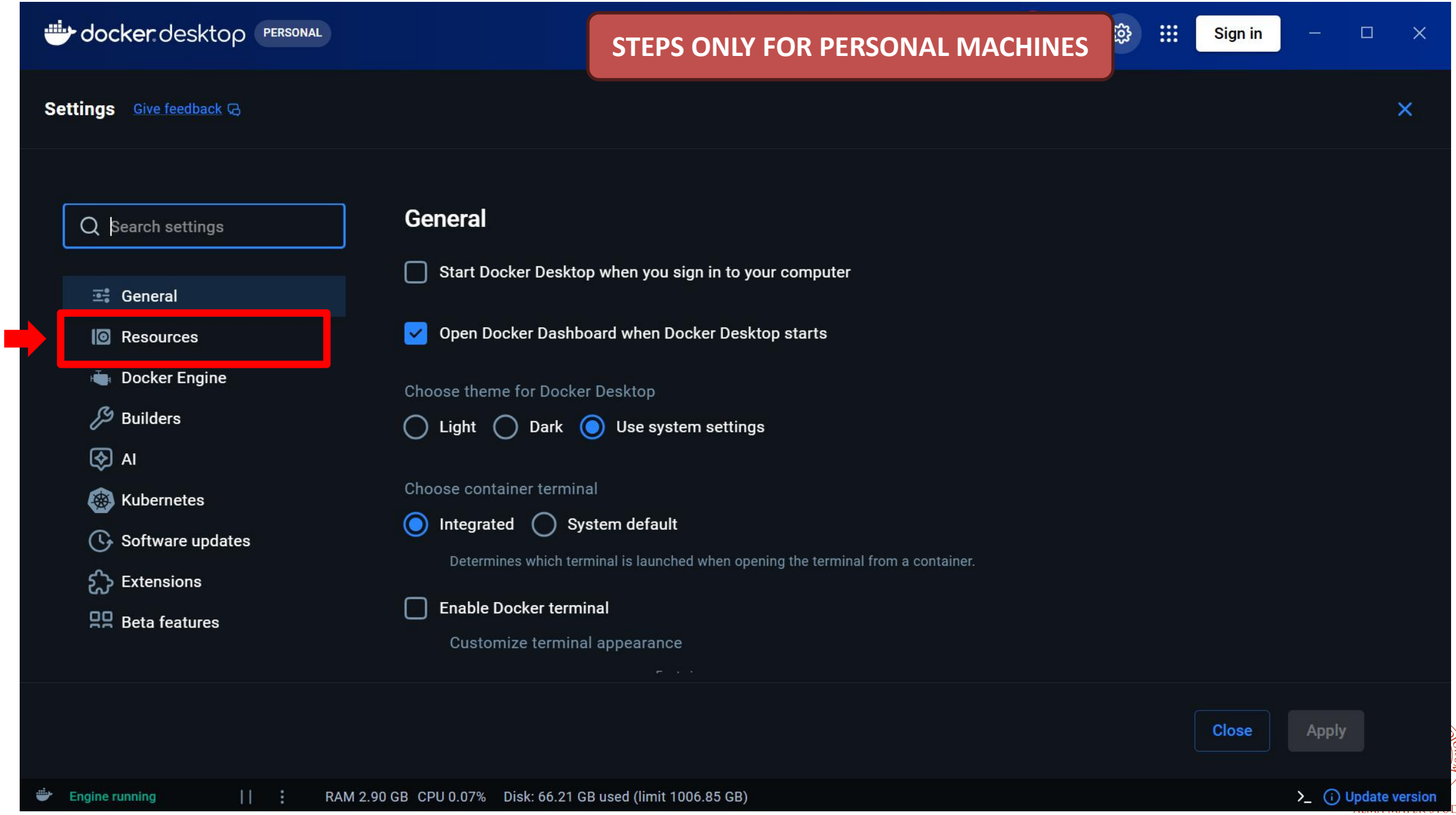


If this error or a similar one pops up, navigate to Docker Desktop.

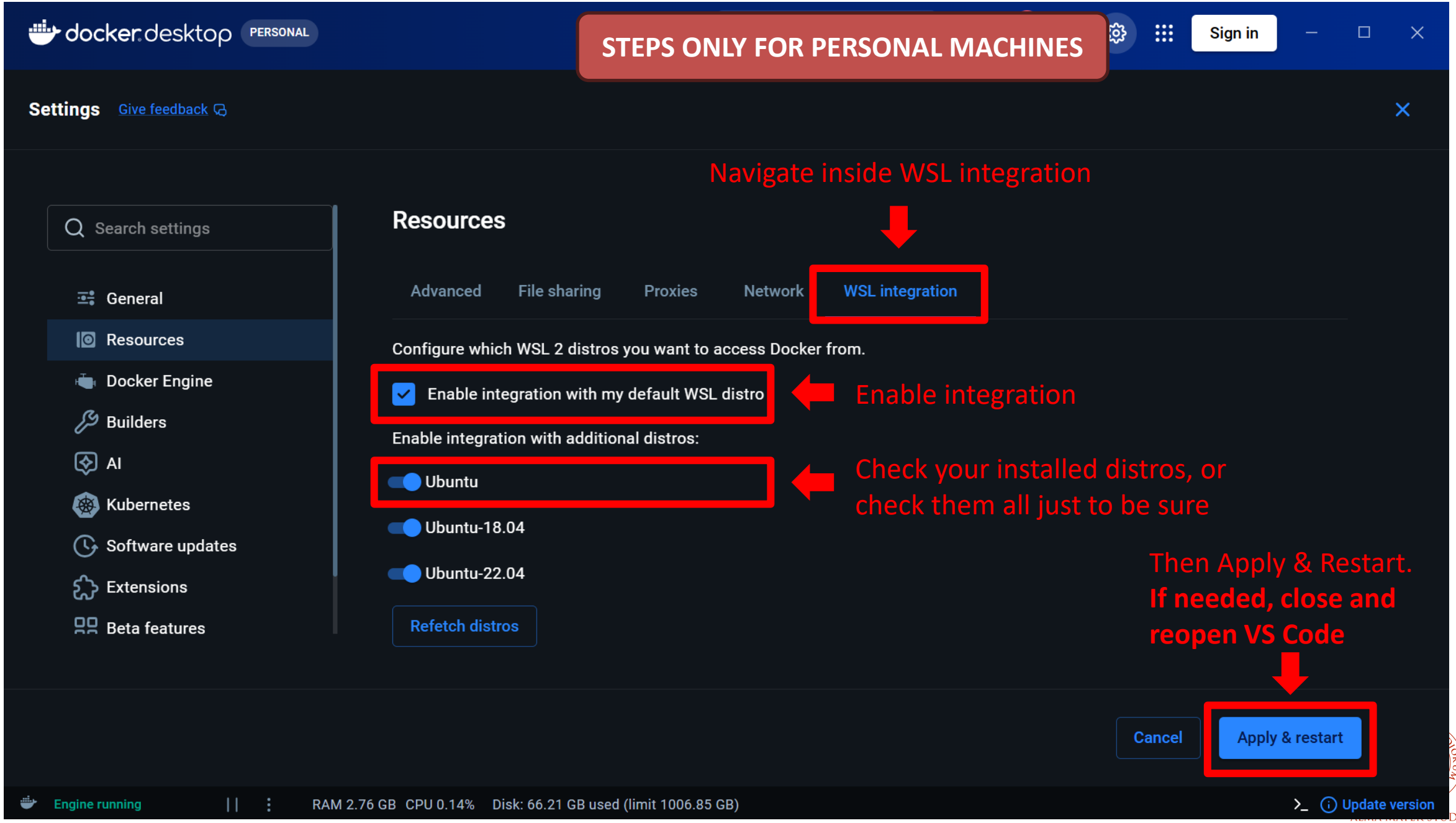
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

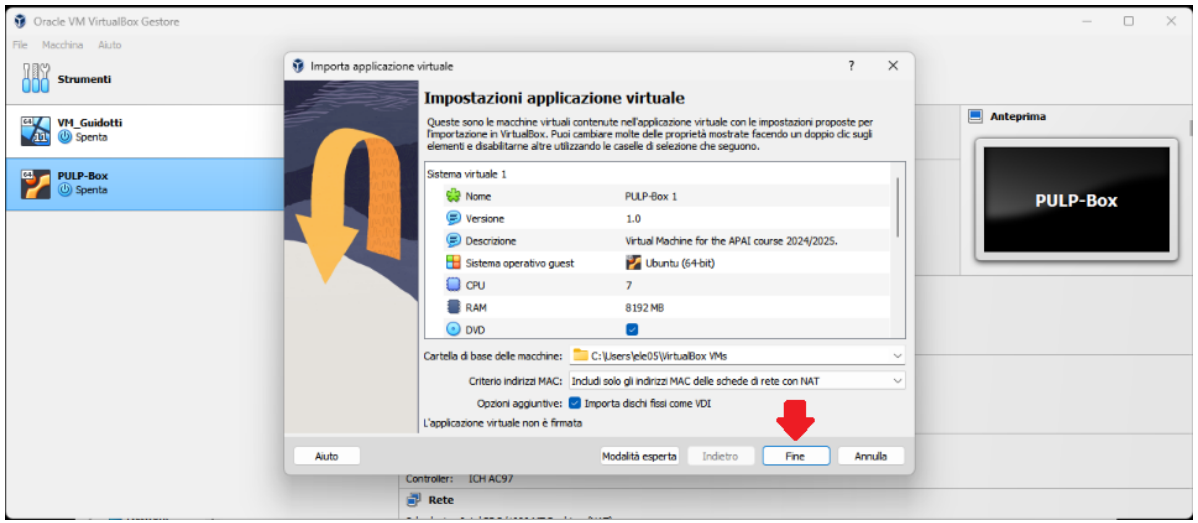
02. Prerequisites – Virtual Machine – Part 1

STEPS ONLY FOR LAB MACHINES

- When using the **lab computers**, you will find an Ubuntu-based virtual machine that already contains everything that is needed for solving the assignments:
  - **PULP-SDK –** so we don't need the Docker container anymore
  - **VS-Code –** in this case, only used for code editing and accessing the terminal
- You will only be required to **download the resources** for the lab, which are also **on Virtuale** (the only website accessible from the lab computers) as a **zip file**. This must be done from a browser **OUTSIDE of the virtual machine**. This is also a good step to **unzip** it, with a simple *right-click -> Extract*

Once you have this zip file, follow these steps:

- **Step 1**: Go to *C:/VM_APAI* and double-click *PULP-box.ova*. When VirtualBox opens, click on *"Finish"/"Fine"*

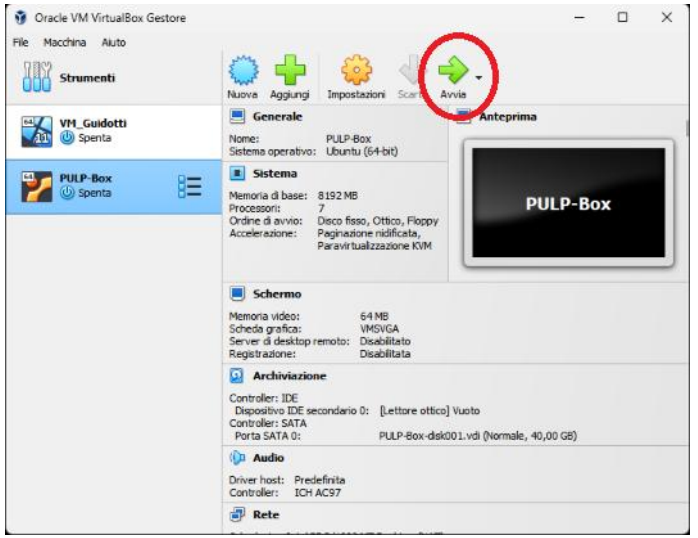


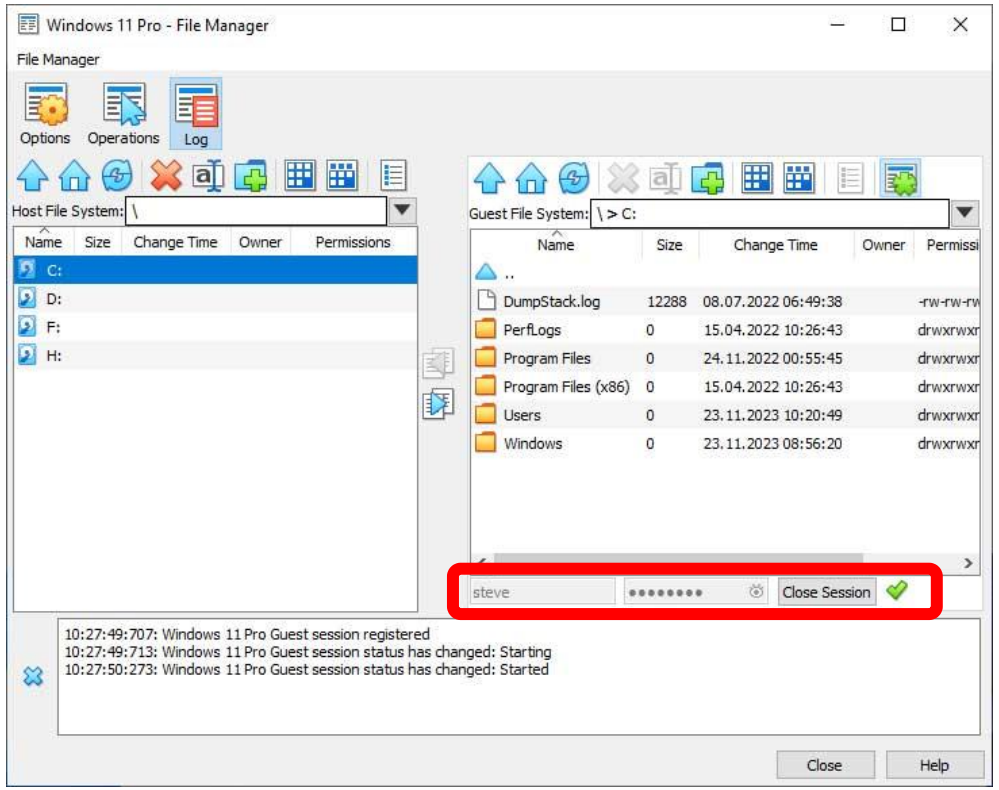
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Virtual Machine – Part 2

STEPS ONLY FOR LAB MACHINES

- **Step 2**: Once the VM is imported, open it by clicking on *"Start"/"Avvia"*
- **Step 3**: For login – the username is ***pulp-box*** and password is ***pulp***
- **Step 4**: Access the files of the VM:
  - From the **top-bar menu**, access ***Machine -> File Manager***
  - In the **left** panel, access files from the **PC**, in the **right**, access files from the **VM**
  - In the **bottom of the right panel**, you will have to **enter the login credentials**





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Virtual Machine – Part 3

STEPS ONLY FOR LAB MACHINES

- **Step 5**: With a simple drag-and-drop, **move the files from the PC location into a VM** location you choose (it must be a subdirectory of the home directory to have file writing rights)
- **Step 6**: Inside the VM, open a terminal and set up the PULP SDK by running

```
source setup_pulp_sdk.sh
```

- **Step 7**: Go to the VM location you chose to put the assignment data in, using the *cd* command in the terminal

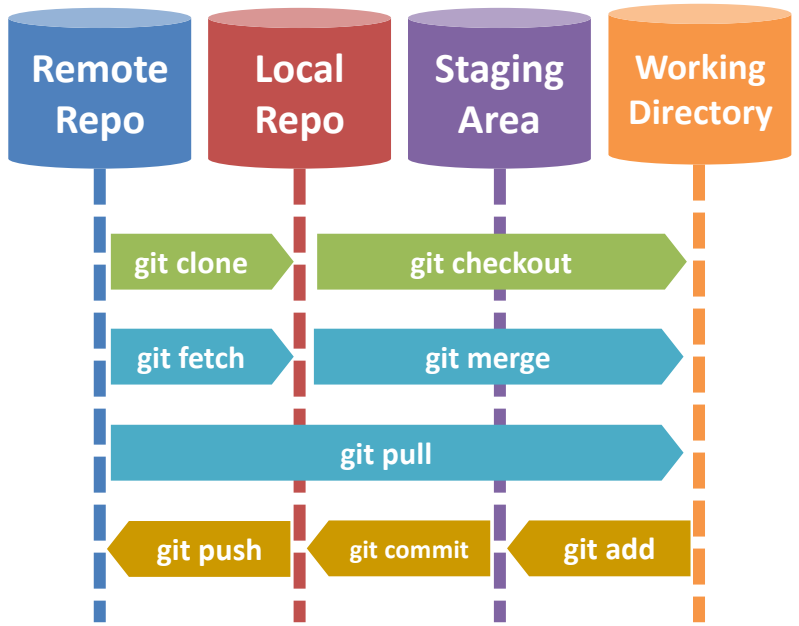
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – git

Link to a useful intro git tutorial



- **What is Git?**
  - A free, open-source version control system that tracks changes to files over time.
  - It lets multiple people work on the same codebase (repository) and merge changes (commits) without overwriting each other's work.
  - It keeps a full history of changes, so you can review, compare, or revert to any earlier version.
- **Common commands we will use**
  - **git clone –** Downloads a full copy of a remote repository to your machine
  - **git checkout** – Switches your working directory to a different branch or commit
  - **git fetch –** Checks the remote for new changes without applying them locally
  - **git merge –** Combines changes from one branch into another
  - **git pull –** Fetches (*downloads)* remote changes and merges them into your local branch (equivalent to *git fetch* + *git merge*)
  - **git add [file1, file2, …] –** Stages (*marks*) the specified files to be included in the next commit
  - **git commit –m "commit message" –** Saves the staged changes to your local repository's history with a message
  - **git push –** Uploads your committed changes to the remote repository



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

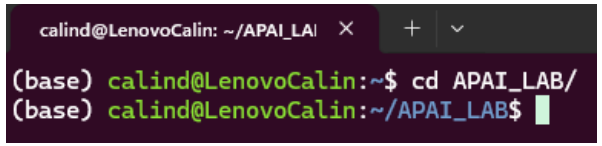
02. Prerequisites – Hello, World Example – Part 1

STEPS ONLY FOR PERSONAL MACHINES

In order to confirm that everything was installed correctly, we will now run the ***Hello, World*** basic example available in the repository of this lab.

The **example is run on WSL, on a Windows** machine, but it should work identically on other operating systems.

- **Step 1:** In a terminal, open the location where you wish to work



- **Step 2:** Clone the repository of lab 5 locally

```
git clone https://github.com/EEESlab/APAI26-LAB05-TinyTransformers
```

- **Step 3:** Enter the cloned directory

```
cd APAI26-LAB05-TinyTransformers/
```

- **Step 4:** Open the directory in VS Code

```
code .
```

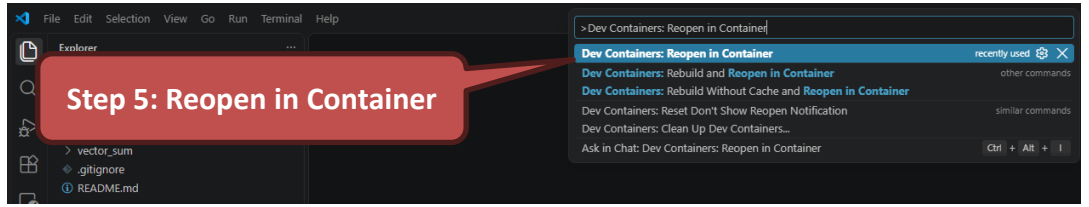

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Hello, World Example – Part 2

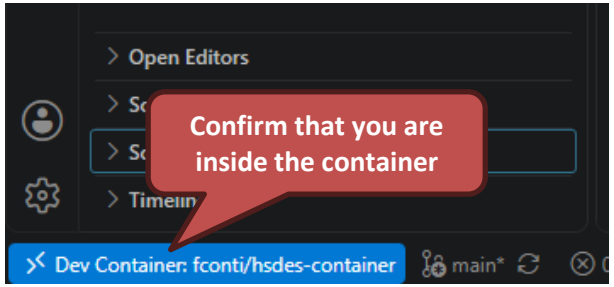
STEPS ONLY FOR PERSONAL MACHINES

* From this point on, we will only work from VS Code, so it is ok to close the terminal you have previously opened.

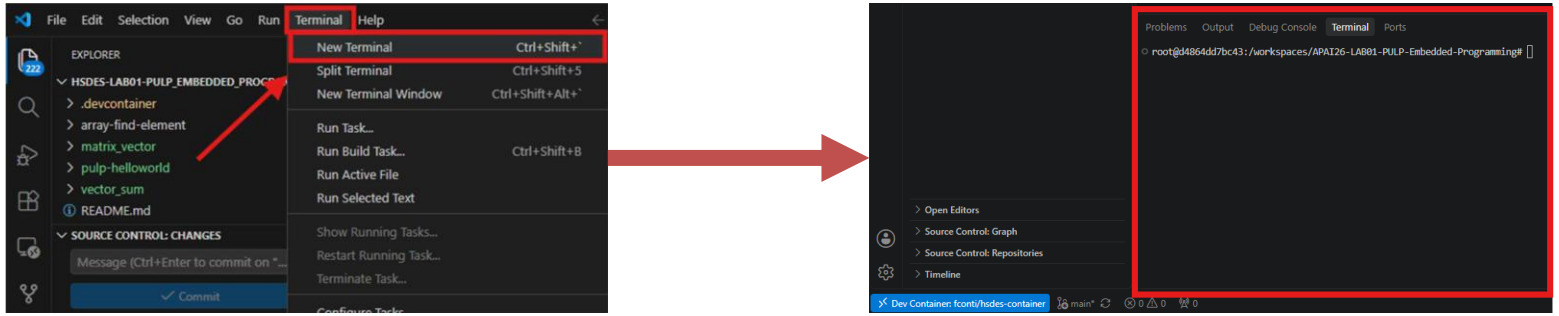
- **Step 5:** At this point, the project is open in the local *environment*. We need to **reopen inside the Docker container**.
  - **Step 5.1**: Press **CTRL** + **Shift** + **P**
  - **Step 5.2**: Type in ***"Dev Containers: Reopen in Container"***
  - **Step 5.3**: Press **Enter**



* We are now **inside the container**. From now on, when opening VS Code, the project will **automatically open inside the Docker Container**. It is confirmed in the bottom-left corner of VS Code.



- **Step 6:** Open a **terminal inside VS Code** (which will be **inside the container**)
  - Access the terminal through the **top-left task bar (*Terminal -> New terminal*)**
  - The terminal **will open in the bottom** of the window.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



03. Introductory Information

03. Intro Information – The Transformer Architecture

- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017): a sequence model built only on **attention**, with **NO recurrence or convolution**

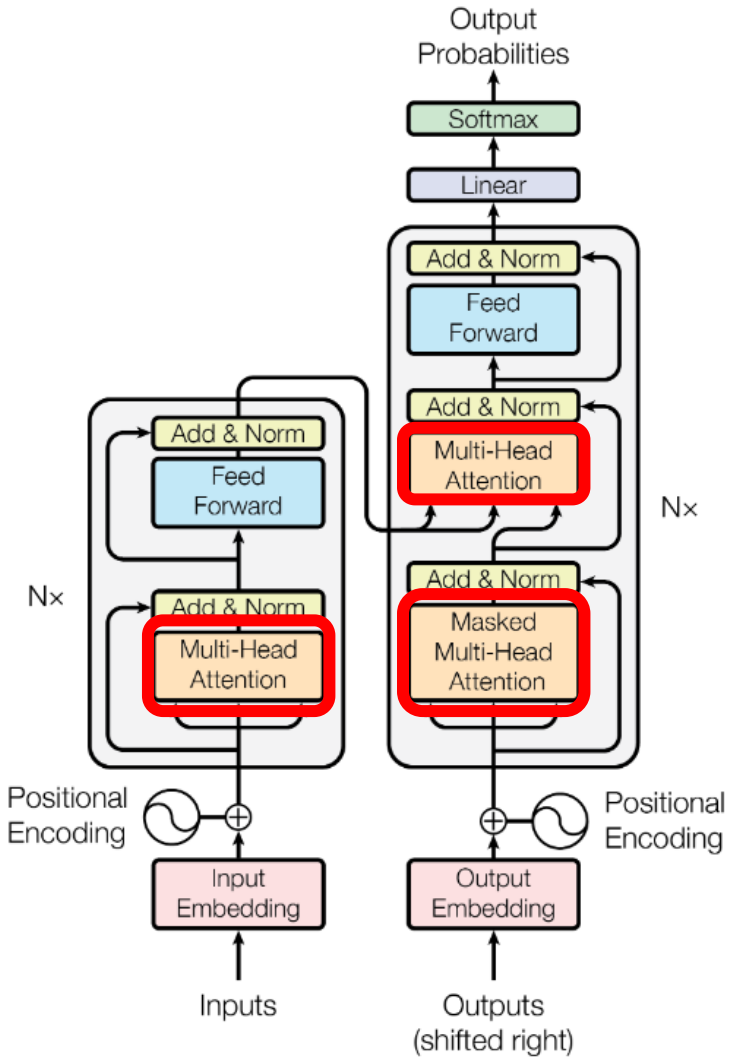
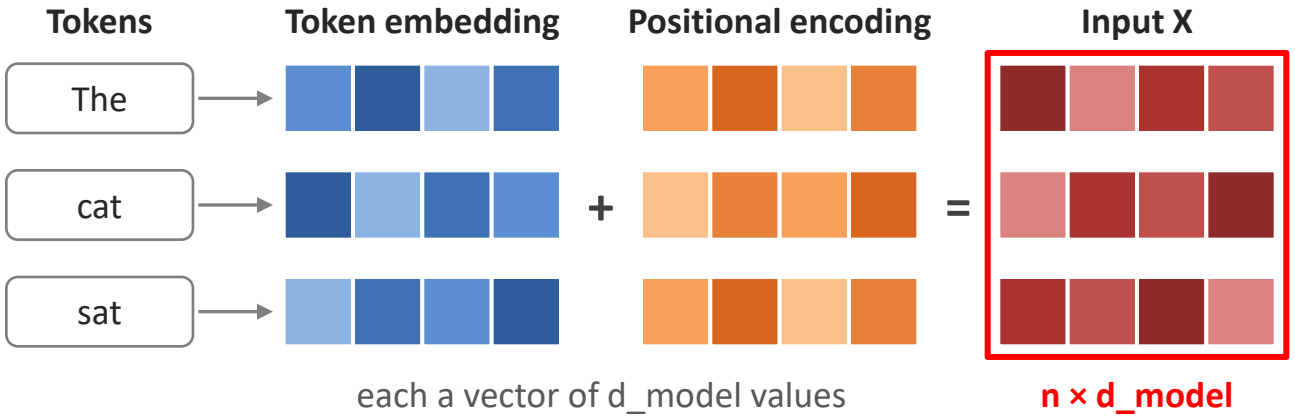


Figure: Transformer architecture, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Transformer Architecture

- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017): a sequence model built only on **attention**, with no recurrence or convolution
- **Input:** token embeddings summed with a positional encoding



Each token becomes one row of X, with d_model elements; the paper uses d_model = 512.

*** Differences from this lab's implementation**:

- d_model is called dim, with dim = 64
- N is called n_layers, and it's set to 5 instead of 6
- No positional encoding, since we use RoPE (Rotary Position Embedding)
- Check ***Tokenizer.c*** to understand how it works

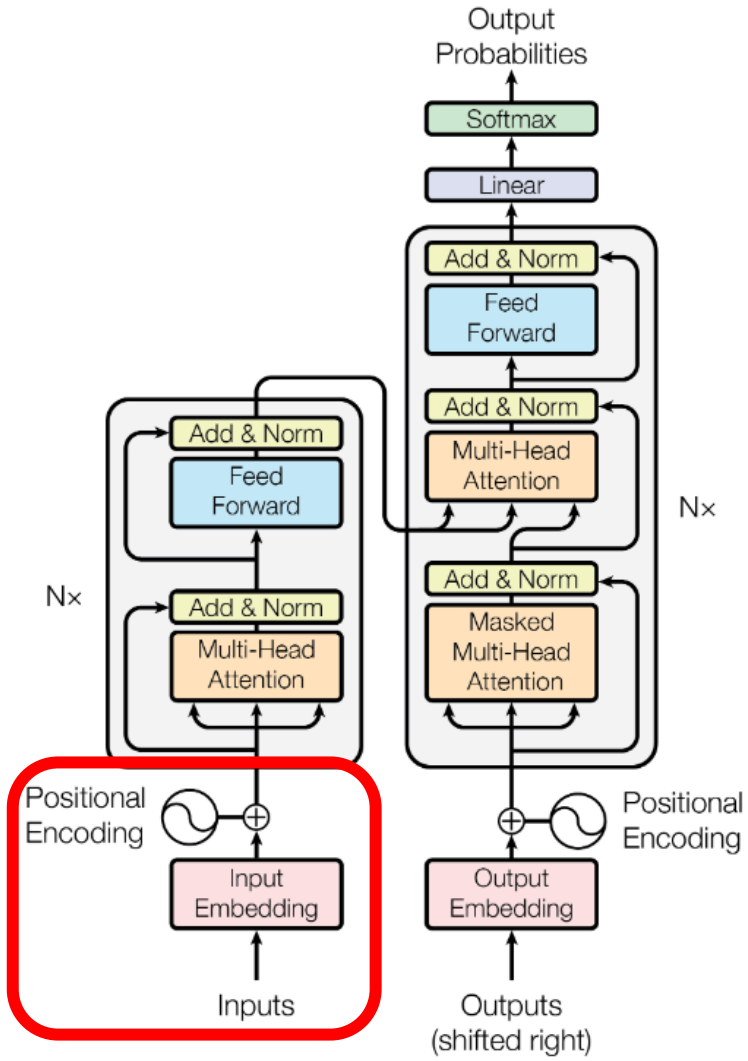


Figure: Transformer architecture, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Transformer Architecture

- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017): a sequence model built only on **attention**, with no recurrence or convolution
- **Input:** token embeddings summed with a positional encoding
- **Encoder–decoder structure**:
  - **Encoder: N blocks** of **self-attention** + feed-forward network/multi-layer perceptron (**FFN/MLP**) turn the input sequence into **contextual representations**

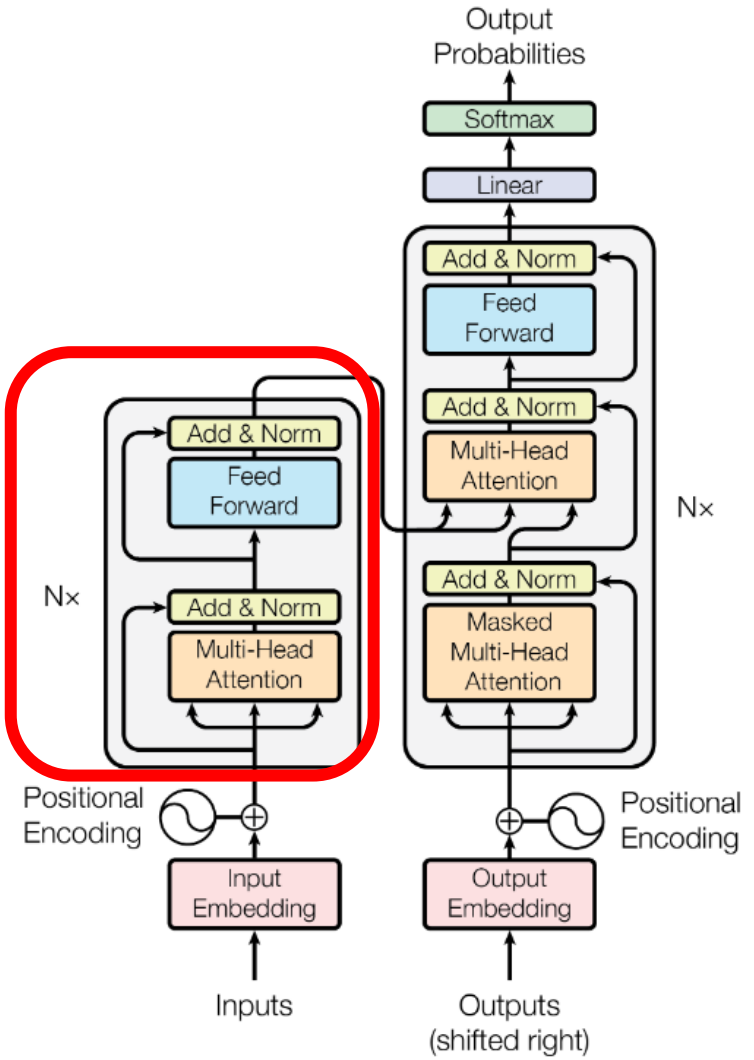


Figure: Transformer architecture, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Transformer Architecture

- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017): a sequence model built only on **attention**, with no recurrence or convolution
- **Input:** token embeddings summed with a positional encoding
- **Encoder–decoder structure**:
  - **Encoder:** N blocks of self-attention + feed-forward network/multi-layer perceptron (FFN/MLP) turn the input sequence into contextual representations
  - **Decoder: N blocks** of **masked self-attention**, applied on previous outputs, **cross-attention** over the **encoder output**, and an **FFN**

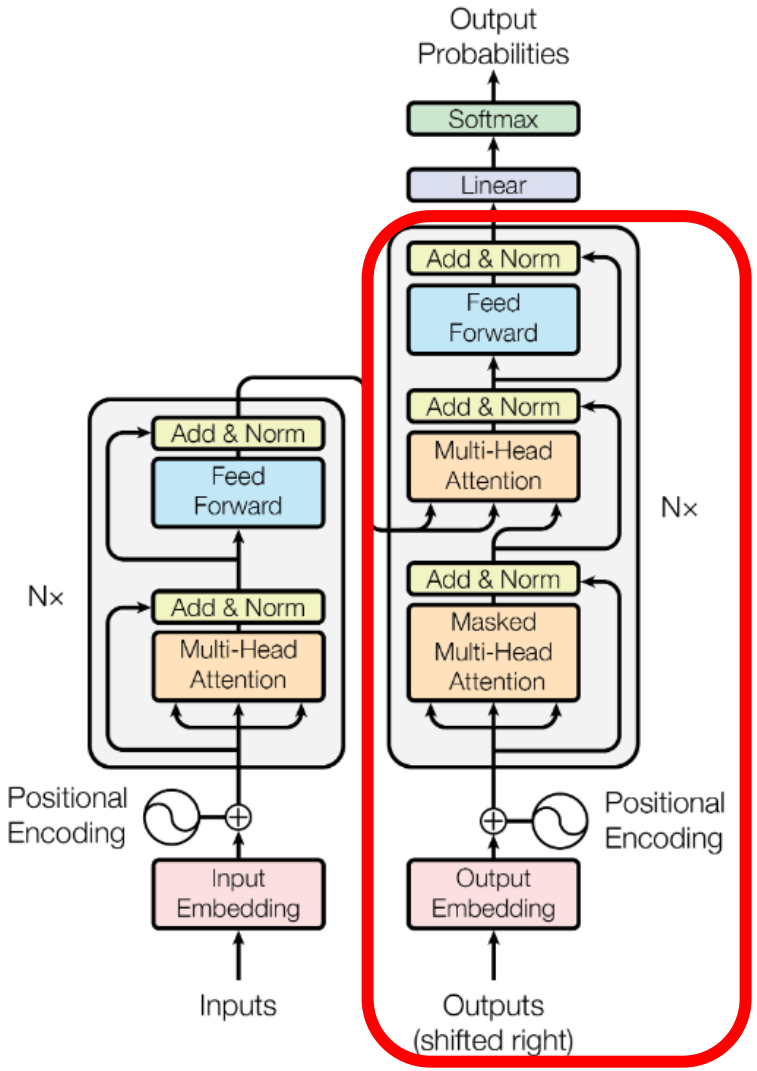
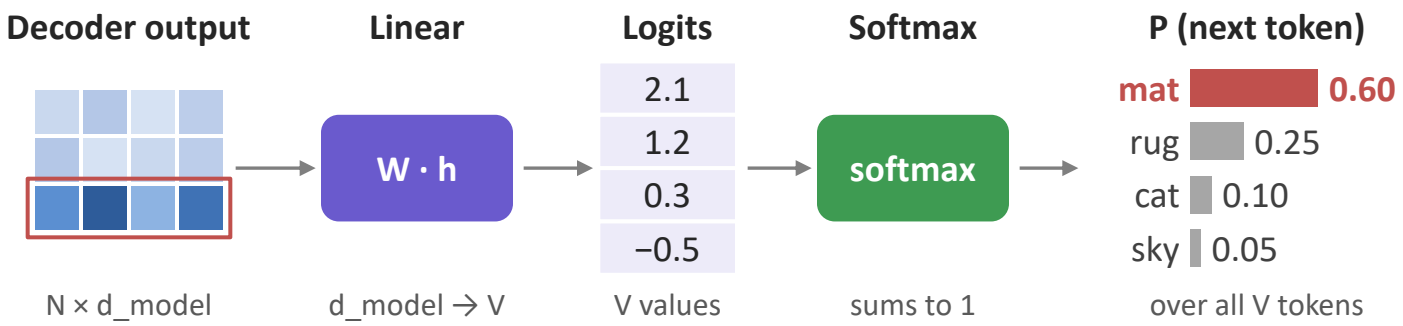


Figure: Transformer architecture, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Transformer Architecture

- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017): a sequence model built only on **attention**, with no recurrence or convolution
- **Input:** token embeddings summed with a positional encoding
- **Encoder–decoder structure**:
  - **Encoder:** N blocks of self-attention + feed-forward network/multi-layer perceptron (FFN/MLP) turn the input sequence into contextual representations
  - **Decoder:** N blocks of masked self-attention, applied on previous outputs, cross-attention over the encoder output, and an MLP
  - **Output:** the **decoder generates one token at a time** (autoregressive); **Linear + softmax give next-token probabilities**



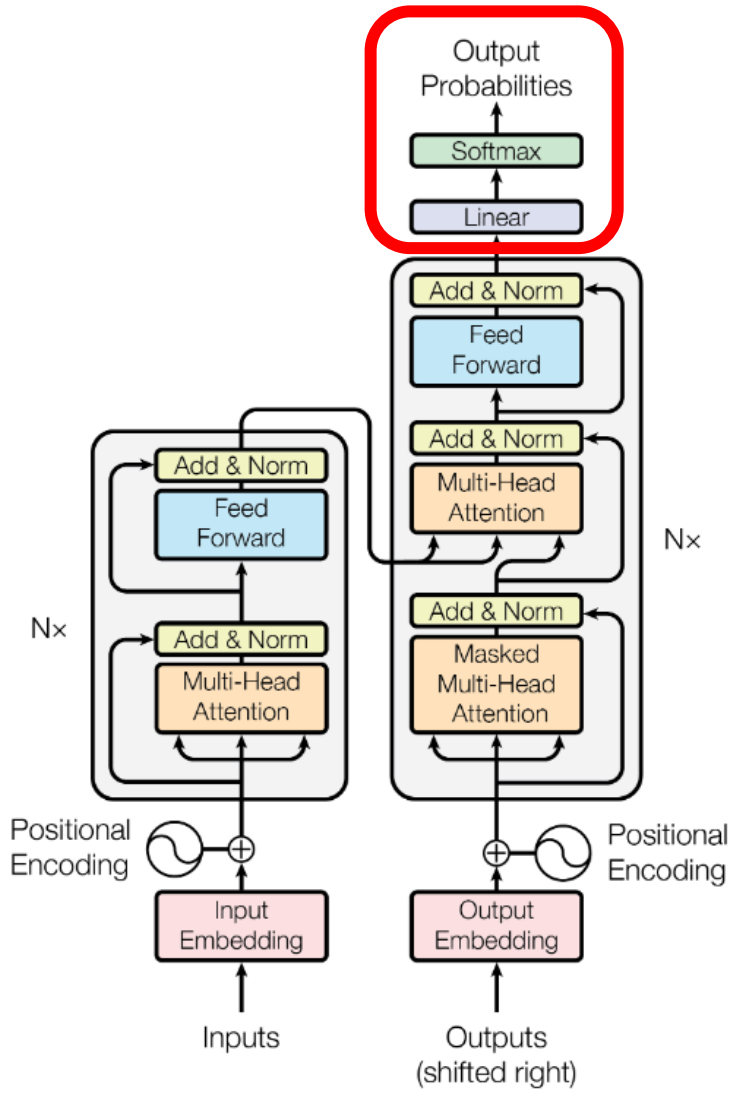


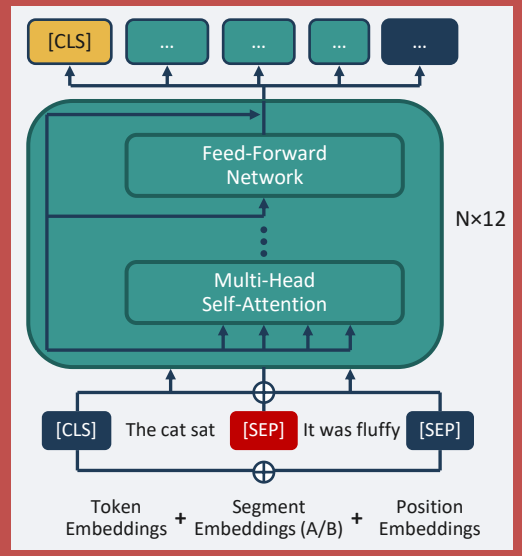
Figure: Transformer architecture, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

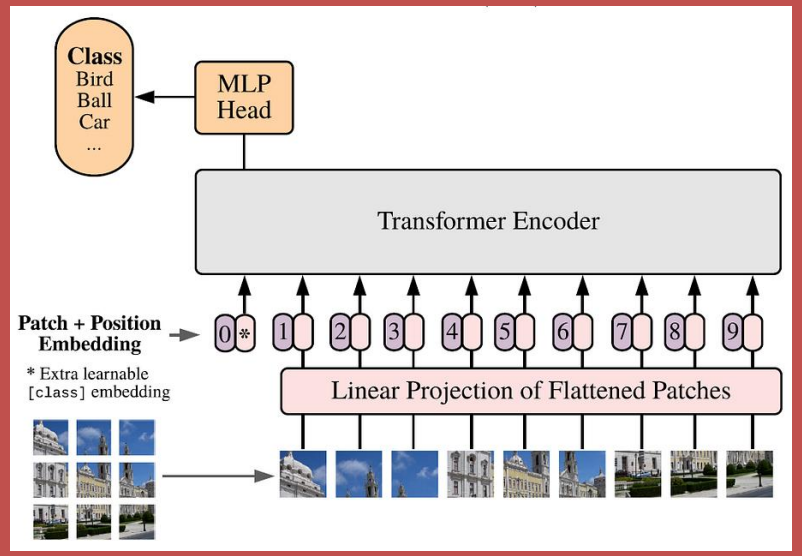
03. Intro Information – The Transformer Architecture

There are variants that keep only half:

Encoder-only

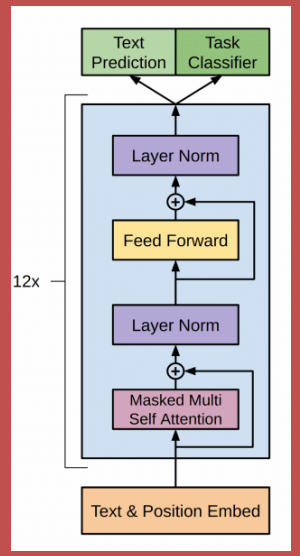


BERT

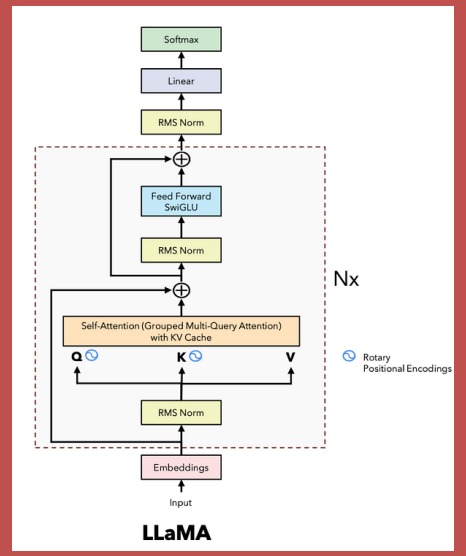


ViT

Decoder-only



GPT



Llama

Usually with a light head on top of the backbone, for sentence-level or word-level classification

For computer vision problems, more recently used in combination with convolutional layers

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Transformer Architecture

- Introduced in *"Attention Is All You Need"* (Vaswani et al., 2017): a sequence model built only on **attention**, with no recurrence or convolution
- **Input:** token embeddings summed with a positional encoding
- **Encoder–decoder structure**:
  - **Encoder:** N blocks of self-attention + feed-forward network/multi-layer perceptron (FFN/MLP) turn the input sequence into contextual representations
  - **Decoder:** N blocks of masked self-attention, applied on previous outputs, cross-attention over the encoder output, and an MLP
  - **Output:** the decoder generates one token at a time (autoregressive); Linear + softmax give next-token probabilities
  - **Variants keep one half:** encoder-only (BERT, ViT) or **decoder-only** (GPT, **Llama**)
- **Add & Norm:** every sub-layer has a **residual connection and LayerNorm**
  - **Residual:** keeps a direct path for the signal and gradients, so deep stacks benefit from increased training stability
  - **LayerNorm:** rescales each token's vector to zero mean and unit variance, keeping activations in a stable range

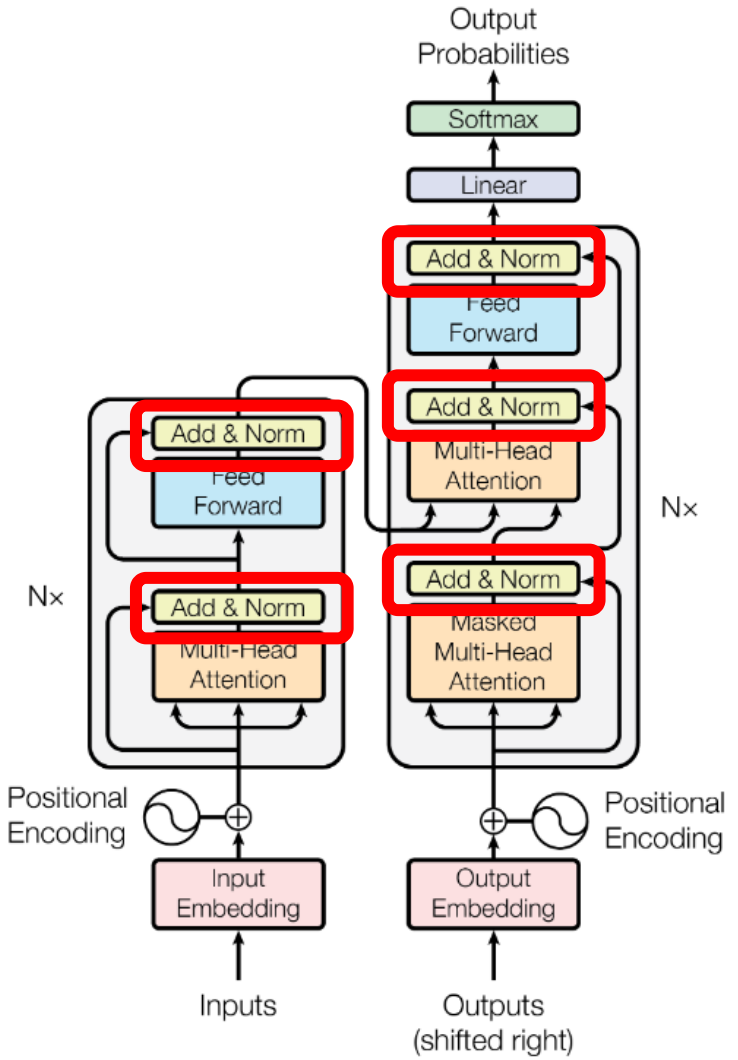


Figure: Transformer architecture, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Attention Mechanism

- **Self-attention:** each token's output is a weighted mix of all tokens in the sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  - **QK^T** scores query–key similarity; scaling by $\sqrt{d_k}$ keeps the softmax stable; **softmax** turns scores into weights that sum to 1

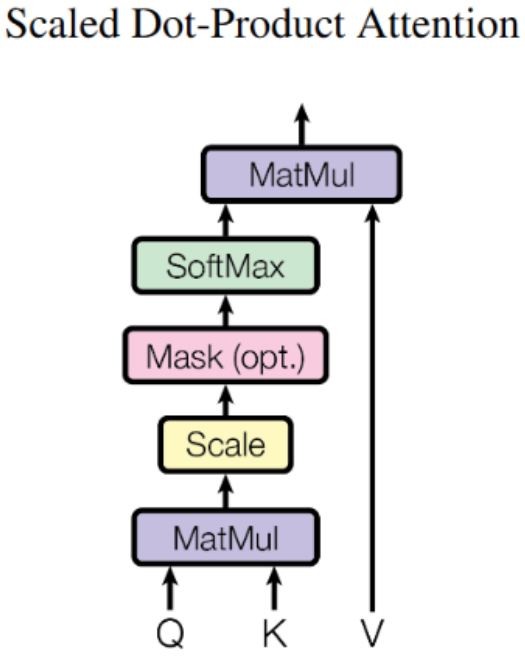


Figure: Scaled Dot-Product Attention, from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Attention Mechanism

- **Self-attention:** each token's output is a weighted mix of all tokens in the sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  - **QK^T** scores query–key similarity; scaling by $\sqrt{d_k}$ keeps the softmax stable; **softmax** turns scores into weights that sum to 1

- **Multi-head attention:** h heads run attention in parallel on smaller projections, then their outputs are concatenated and projected back

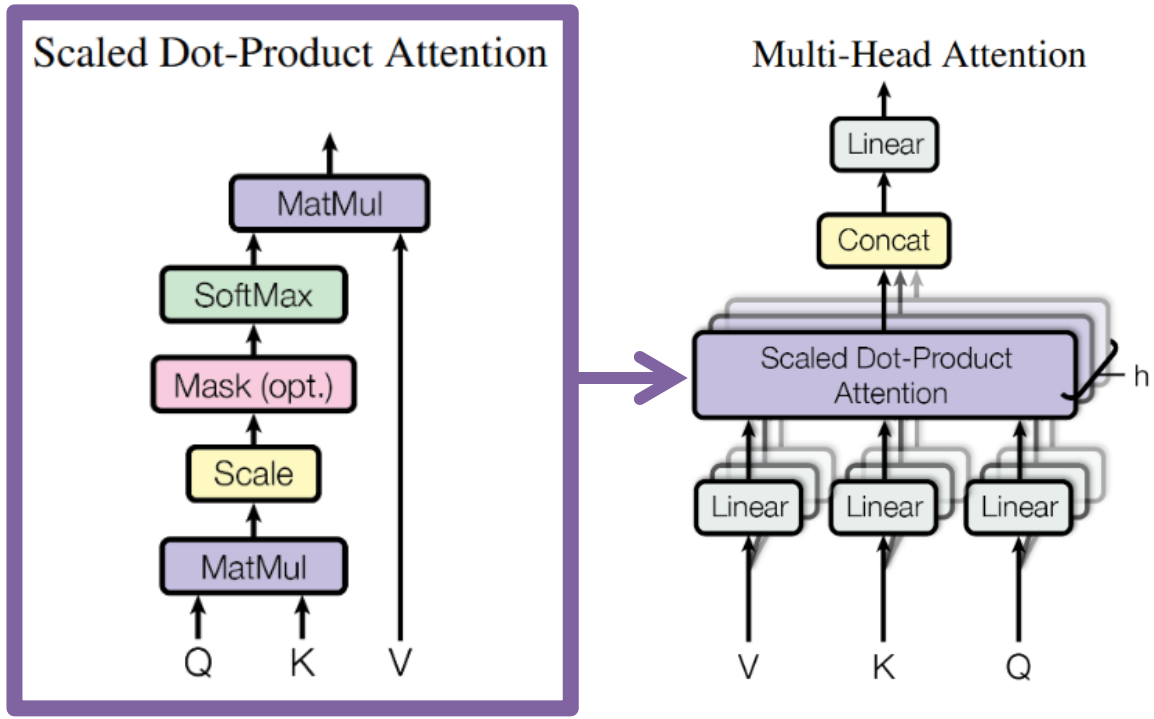


Figure: Scaled Dot-Product Attention (left) and Multi-Head Attention (right), from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Attention Mechanism

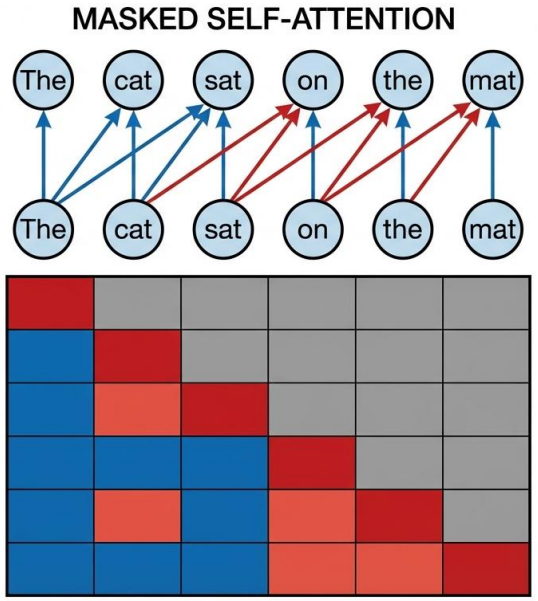
- **Self-attention:** each token's output is a weighted mix of all tokens in the sequence:

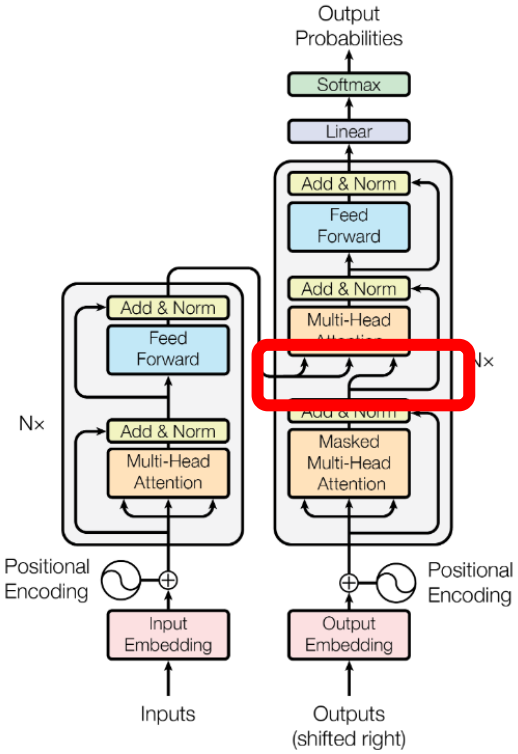
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  - **QK^T** scores query–key similarity; scaling by $\sqrt{d_k}$ keeps the softmax stable; **softmax** turns scores into weights that sum to 1

- **Multi-head attention:** h heads run attention in parallel on smaller projections, then their outputs are concatenated and projected back

- **Attention in the decoder:**
  - **Masked self-attention:** a mask hides future tokens, so each position attends only to earlier ones
  - **Cross-attention:** Q comes from the decoder, K and V from the encoder output





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – The Attention Mechanism

- **Self-attention:** each token's output is a weighted mix of all tokens in the sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  - **QK^T** scores query–key similarity; scaling by $\sqrt{d_k}$ keeps the softmax stable; **softmax** turns scores into weights that sum to 1
- **Multi-head attention:** h heads run attention in parallel on smaller projections, then their outputs are concatenated and projected back
- **Attention in the decoder:**
  - **Masked self-attention:** a mask hides future tokens, so each position attends only to earlier ones
  - **Cross-attention:** Q comes from the decoder, K and V from the encoder output
- **Cost:** the **N × N score matrix** makes compute and memory grow **quadratically** with **sequence length N**

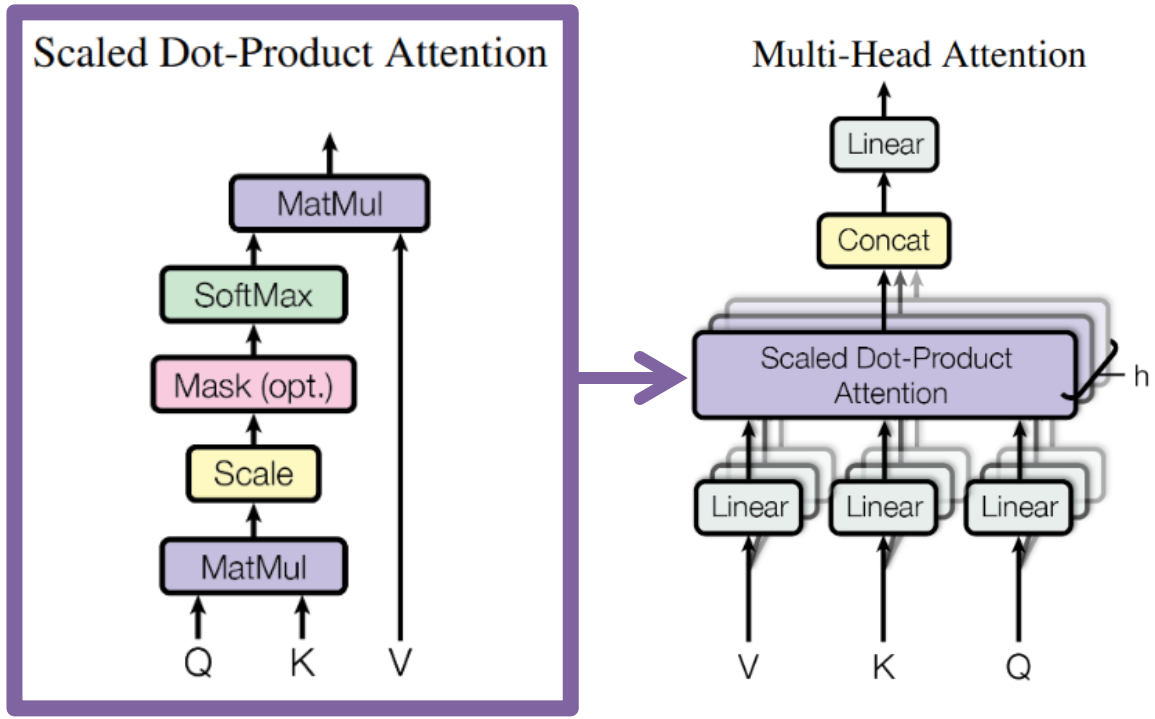
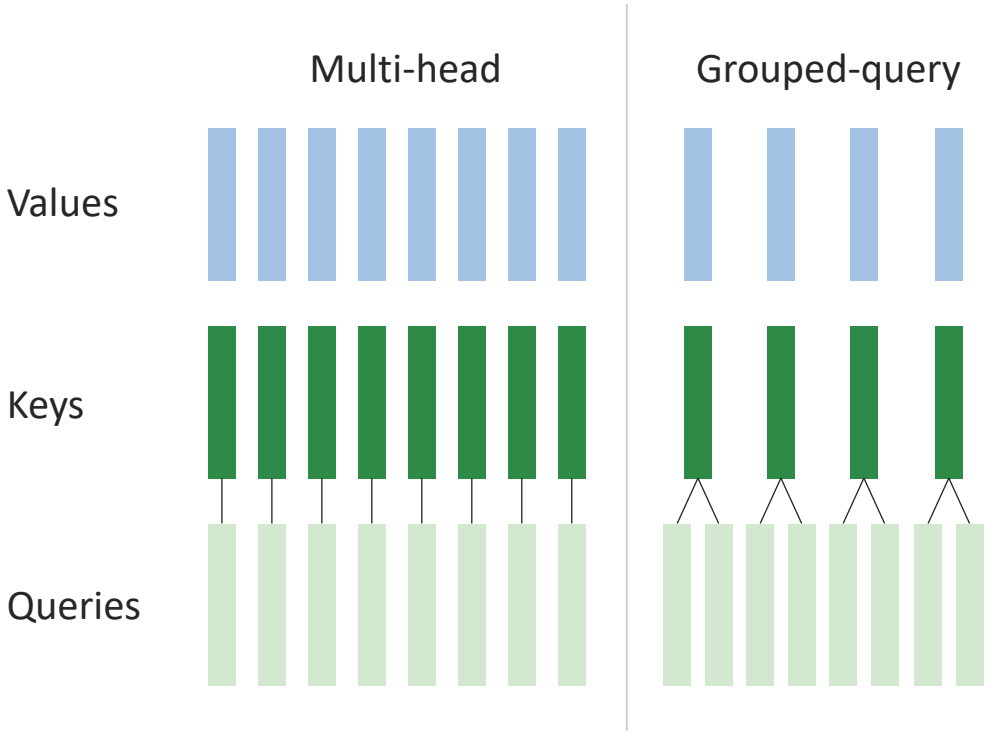


Figure: Scaled Dot-Product Attention (left) and Multi-Head Attention (right), from Vaswani et al., "Attention Is All You Need", 2017

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Grouped-Query Attention

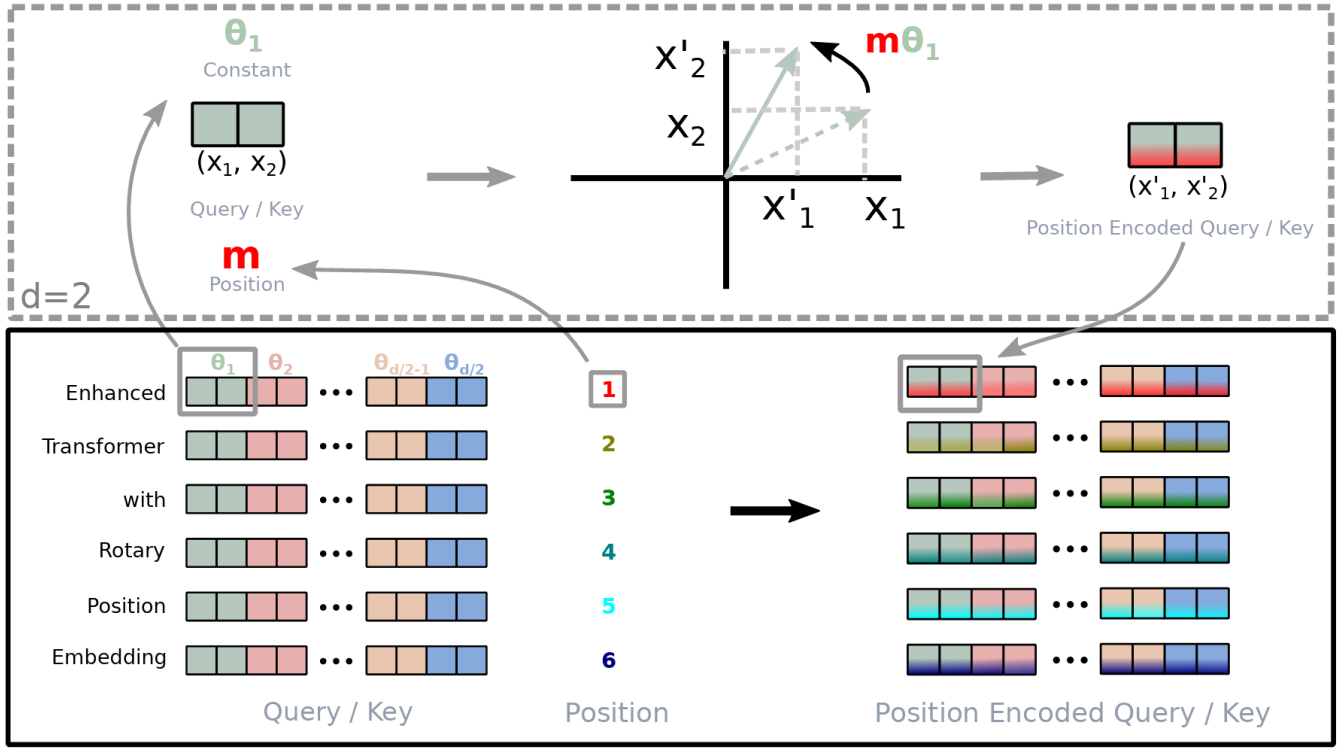
- **Llama 2**, the architecture on which this lab is based, has several particularities compared to the original transformer design, besides being decoder-only
- **Grouped-Query Attention (GQA)** (Llama 34B, 70B, and TinyStories260k): each of the heads has its own query, while groups of N^H/N^{KV} heads share one key and value head
    - **The benefit** is that the K/V weights and the KV cache are a fraction of the size they would be with standard multi-head attention, with **the drawback** of losing KV representation capacity



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: RoPE

- **RoPE (Rotary Position Embedding)** replaces default positional encodings:
  - **Rotates pairs of elements** of Q and K by an **angle that depends on the token position**
  - Attention scores then depend on the **relative distance** between tokens



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Sampling Algorithm

Toy example: 6-token vocabulary (the real model has 512), prompt “Lily went to the park with her …”, top-p = 0.9

Step	dog	cat	ball	tree	car	sky
Logits	2.0	1.5	1.0	0.5	0.0	−1.0

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Sampling Algorithm

Toy example: 6-token vocabulary (the real model has 512), prompt “Lily went to the park with her …”, top-p = 0.9

Step	dog	cat	ball	tree	car	sky
Logits	2.0	1.5	1.0	0.5	0.0	−1.0
÷ temperature (T = 0.9)	2.22	1.67	1.11	0.56	0.00	−1.11

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Sampling Algorithm

Toy example: 6-token vocabulary (the real model has 512), prompt “Lily went to the park with her …”, top-p = 0.9

Step	dog	cat	ball	tree	car	sky
Logits	2.0	1.5	1.0	0.5	0.0	−1.0
÷ temperature (T = 0.9)	2.22	1.67	1.11	0.56	0.00	−1.11
Softmax	44.7%	25.7%	14.7%	8.5%	4.9%	1.6%

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Sampling Algorithm

Toy example: 6-token vocabulary (the real model has 512), prompt “Lily went to the park with her …”, top-p = 0.9

Step	dog	cat	ball	tree	car	sky
Logits	2.0	1.5	1.0	0.5	0.0	−1.0
÷ temperature (T = 0.9)	2.22	1.67	1.11	0.56	0.00	−1.11
Softmax	44.7%	25.7%	14.7%	8.5%	4.9%	1.6%
Top-p: cumulative	0.447	**0.704**	0.851	0.936	outside	outside

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Sampling Algorithm

Toy example: 6-token vocabulary (the real model has 512), prompt “Lily went to the park with her …”, top-p = 0.9

Step	dog	cat	ball	tree	car	sky
Logits	2.0	1.5	1.0	0.5	0.0	−1.0
÷ temperature (T = 0.9)	2.22	1.67	1.11	0.56	0.00	−1.11
Softmax	44.7%	25.7%	14.7%	8.5%	4.9%	1.6%
Top-p: cumulative	0.447	0.704	0.851	0.936	outside	outside
Renormalization	[0, 0.478)	[0.478, 0.752)	[0.752, 0.910)	[0.910, 1)	—	—

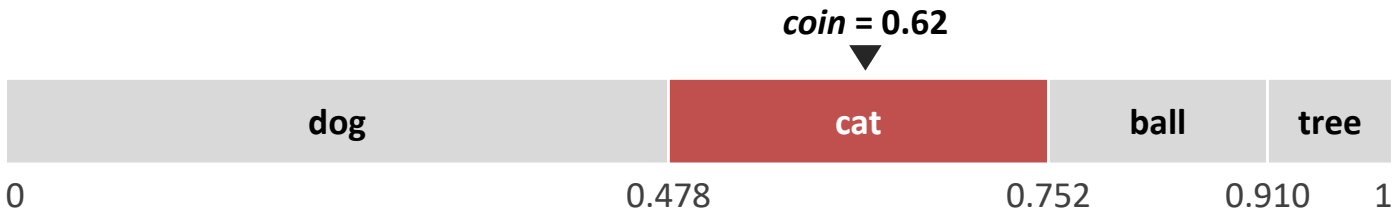
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: Sampling Algorithm

Toy example: 6-token vocabulary (the real model has 512), prompt “Lily went to the park with her …”, top-p = 0.9

Step	dog	cat	ball	tree	car	sky
Logits	2.0	1.5	1.0	0.5	0.0	−1.0
÷ temperature (T = 0.9)	2.22	1.67	1.11	0.56	0.00	−1.11
Softmax	44.7%	25.7%	14.7%	8.5%	4.9%	1.6%
Top-p: cumulative	0.447	**0.704**	0.851	0.936	outside	outside
Renormalization	[0, 0.478)	**[0.478, 0.752)**	[0.752, 0.910)	[0.910, 1)	—	—

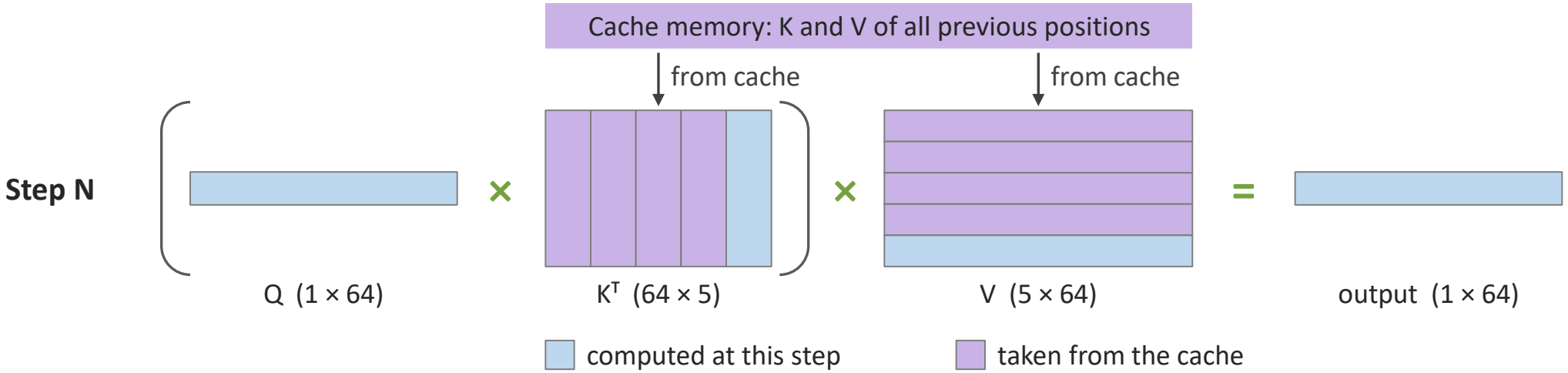
Slice chosen with a **random value generated between 0 and 1**



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – Llama 2: KV Caching

- **KV cache** is an **optimization** to reduce the number of computations during the **autoregressive phase**.
- During this phase, which is the one in which the model **generates text** related to the input prompt, the model will start generating **a single output token per iteration**, based on the input and the already generated outputs.
- During every iteration of this phase, without optimizations, the model should **recompute the K and V tensors** and the attention matrix based on **all the previous tokens** in the sequence.
- The KV cache is a matrix that **stores the K and V vectors** already computed for **every previous position**, so at each step only the **K and V of the new token** are computed.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Intro Information – stories260k compared to Llama 2

- In the lab, we use a **similar, but reduced architecture** compared to Llama 2 (**260k parameters**, compared to 7B for smallest Llama 2)

- Dimensions are much reduced: 172 vs. 11k hidden dimension, 5 vs. 32 layers, 8/4 vs. 32/32 **Q/KV heads**, 8 vs. 128 head size, 512 vs. 32k **vocabulary size** with a **custom tokenizer**, 256 vs. 4096 **context** size

- It is trained **only on simple children's stories**, with **no instruction or chat** fine-tuning

- It writes **plausible short stories** but has **no general knowledge**. Its value is that it is **small enough to run on a PULP cluster** while **keeping the real Llama 2 dataflow**.

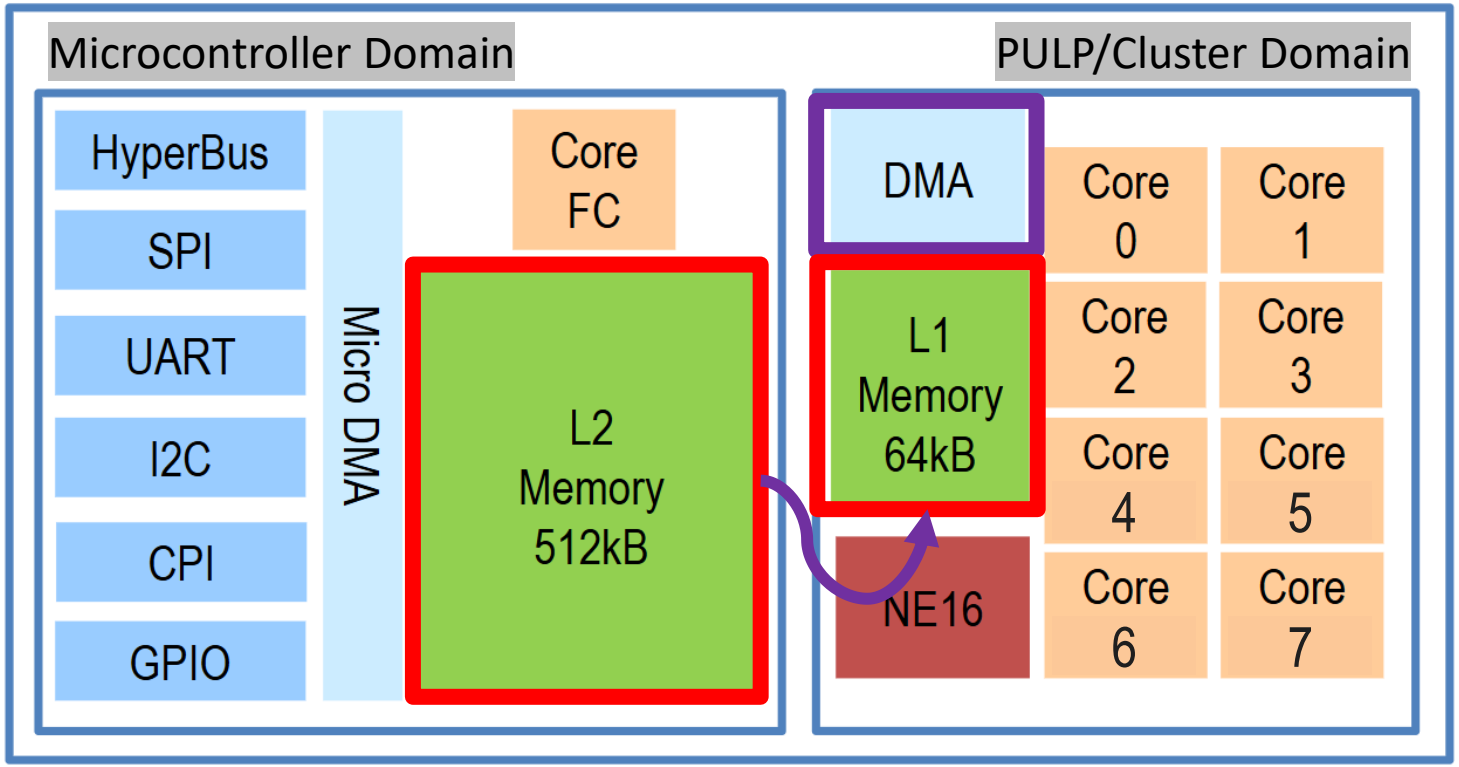
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



04. Assignment

04. Assignment – Task 1 – Attention

- **Task 1.1** – Manually compute sizes of the QKV tensors for a single layer. Use size definitions from conf_and_weights_fp16.h
- **Task 1.2** – Write code for moving the QKV weights into the cluster, from L2 to L1, using the DMA. Follow instructions in the comments, in the code.

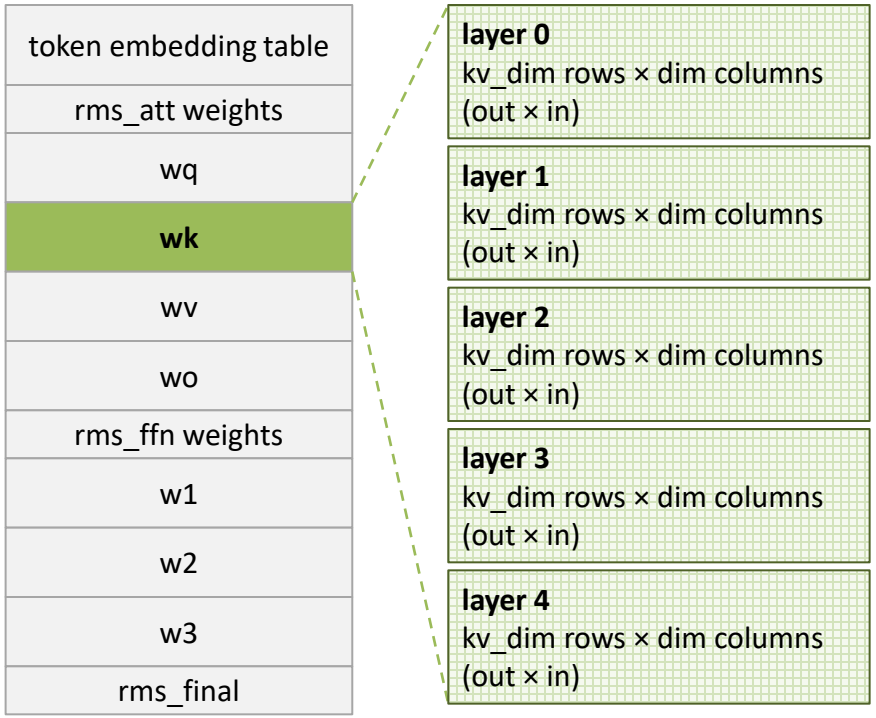


- **Task 1.3** – Write code for computing the attention product. One spot is for loop unrolling, while the other for the leftovers of the unrolling. Follow the instructions in the comments.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignment – Task 1 – L2 and L1 Weight Layout

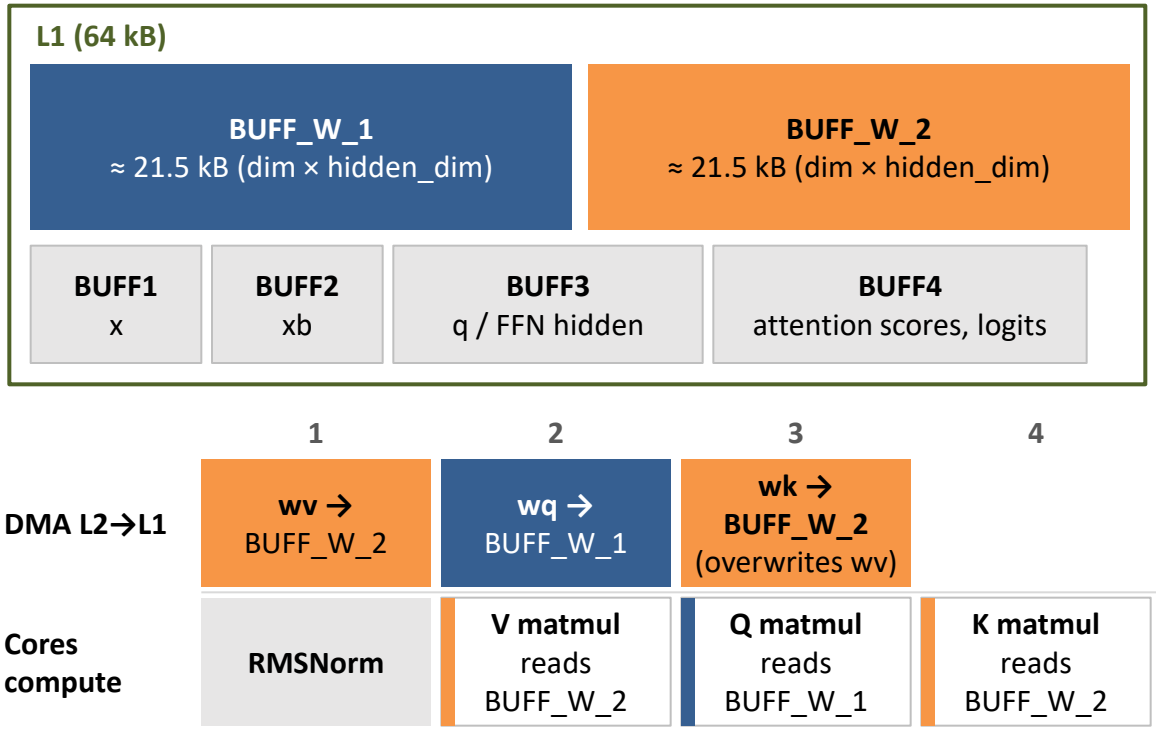
Weights in L2



wcls (classifier) reuses the token embedding table

Each matrix tensor is stored as [layer][out][in]: the layers are back to back, row by row.

Weight buffers in L1



The FFN and the classifier follow the same pattern, alternating BUFF_W_1 and BUFF_W_2.

DMA size is in BYTES (elements × 2 for fp16), and a buffer can only be overwritten once the matmul reading it has finished.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignment – Tasks 2 & 3 – Using the transformer

- **Task 2.1** – Complete the ***net_step()*** function. Look for ***MISSING_FUNCTION*** placeholders and replace them with proper calls. The missing functions should:
  - **tokenize** the prompt
  - do a **forward pass** of the model
  - sample the **next token**
  - and **decode the token**
- **Task 2.2** – **Profile** the attention
  - **Uncomment the -DSTATS flag in the Makefile and add the profiling macros in the proper places**
  - Run ***make clean all run***
  - Answer the questions in the assignment document
- **Task 3 – Fully run the pipeline** and **generate a children's story**. Remember to paste the resulting story in the assignment document. **REMEMBER TO COMMENT BACK THE STATS FLAG!**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Călin Diaconu – calin.diaconu2@unibo.it

Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it

Angelo Anzalone - angelo.anzalone4@unibo.it

Ihor Sachok – ihor.sachok@unibo.it

Luca Parigi – luca.parigi2@unibo.it

Marco La Barbera – marco.labarbera2@unibo.it